

LAÇO DE REPETIÇÃO WHILE EM PYTHON

Material de apoio — Linguagem de Programação (Python)

Objetivo da aula

Compreender como o while repete comandos enquanto uma condição for verdadeira, aprendendo a controlar a repetição com variáveis, condições e atualização de valores.

1. O que é repetição?

Em programação, muitas vezes precisamos executar o mesmo trecho de código várias vezes. Em vez de escrever o mesmo comando repetidamente, usamos um laço de repetição.

O while é usado quando a repetição depende de uma condição. Enquanto essa condição for verdadeira, o bloco será executado.

```
while condição:  
    comando  
    comando
```

Ideia principal: condição verdadeira → repete | condição falsa → para

Pense assim:

PERGUNTA	AÇÃO
A condição é verdadeira?	SIM → executa o bloco novamente.
A condição é falsa?	NÃO → sai do while e continua o programa.

2. A estrutura básica do while

O while precisa de três elementos importantes: uma condição, um bloco de comandos e, na maioria dos casos, uma atualização que faça a condição mudar.

```
contador = 1
```

```
while contador <= 5:  
    print(contador)  
    contador = contador + 1
```

Saída: 1, 2, 3, 4, 5

Entendendo passo a passo

- contador = 1 → começa com 1.
- contador <= 5 → pergunta se ainda deve repetir.
- print(contador) → mostra o valor atual.
- contador = contador + 1 → aumenta o contador.

Quando contador passa de 5, a condição deixa de ser verdadeira e o while termina.

3. O erro mais comum: esquecer de atualizar

```
contador = 1
```

```
while contador <= 5:  
    print(contador)  
    # contador não muda!
```

Nesse caso, a condição continua verdadeira e o programa pode entrar em um loop infinito. Por isso, sempre observe o que faz a condição mudar.

ANTES DO WHILE	DURANTE O WHILE	QUANDO PARA
contador = 1	contador aumenta	contador > 5

4. while com entrada do usuário

Uma aplicação muito comum é pedir uma informação ao usuário e repetir a pergunta até que ele forneça uma resposta válida.

```
senha = ""

while senha != "1234":
    senha = input("Digite a senha: ")

print("Acesso permitido!")
```

Como funciona?

- A variável senha começa vazia.
- O programa verifica se a senha é diferente de 1234.
- Se for diferente, pede novamente.
- Quando o usuário digita 1234, a condição fica falsa e o while termina.

5. while para repetir uma quantidade determinada de vezes

Também podemos usar um contador para controlar quantas vezes algo será executado.

```
quantidade = int(input("Quantos alunos? "))

contador = 1

while contador <= quantidade:
    nome = input("Digite o nome do aluno: ")
    print("Aluno cadastrado:", nome)
    contador = contador + 1
```

Aqui o usuário decide quantas vezes o cadastro será realizado. Esse padrão será muito útil quando começarmos a trabalhar com listas.

Regra prática

Se você sabe...	Pode pensar em...
quantas vezes repetir	contador + while
até quando repetir	condição + while
que precisa validar uma entrada	while + condição

6. while com menu

Um dos exemplos mais importantes para os nossos exercícios é o menu. O programa continua funcionando até o usuário escolher a opção de sair.

```
opcao = 0

while opcao != 3:
    print("\n1 - Cadastrar aluno")
    print("2 - Listar alunos")
    print("3 - Sair")

    opcao = int(input("Escolha uma opção: "))

    if opcao == 1:
        print("Cadastro de aluno")
    elif opcao == 2:
        print("Lista de alunos")
    elif opcao == 3:
        print("Saindo...")
    else:
        print("Opção inválida!")
```

Observe a lógica

- O while mantém o menu funcionando.
- O if / elif / else decide o que fazer em cada opção.
- A opção 3 muda a condição do while para falsa.
- Enquanto o usuário não escolher 3, o menu volta a aparecer.

7. Um exemplo próximo da nossa atividade: cadastro de livros

```
quantidade = int(input("Quantos livros deseja cadastrar? "))

contador = 1

while contador <= quantidade:
    titulo = input("Digite o título do livro: ")
    autor = input("Digite o autor: ")

    print("Livro:", titulo)
    print("Autor:", autor)

    contador = contador + 1

print("Cadastro finalizado!")
```

Importante: neste exemplo ainda não estamos guardando os livros. Estamos apenas praticando a repetição. Depois, com listas, poderemos armazenar os dados.

8. while + if: repetição com decisão

O while pode trabalhar junto com o if. Assim, o programa repete e, a cada repetição, toma uma decisão diferente conforme os dados informados.

```
contador = 1

while contador <= 5:
    idade = int(input("Digite a idade: "))

    if idade >= 18:
        print("Maior de idade")
    else:
        print("Menor de idade")

    contador = contador + 1
```

O while controla quantas pessoas serão analisadas, enquanto o if decide qual mensagem será apresentada para cada pessoa.

9. while para validar dados

Outra utilização muito importante é impedir que o programa continue enquanto uma entrada estiver fora do intervalo esperado.

```
nota = float(input("Digite uma nota de 0 a 10: "))

while nota < 0 or nota > 10:
    print("Nota inválida!")
    nota = float(input("Digite uma nota de 0 a 10: "))

print("Nota registrada:", nota)
```

Perceba o padrão:

1	Recebe o valor
2	Verifica a condição
3	Se estiver errado, pede novamente
4	Quando estiver correto, sai do while

Dica do professor

Antes de executar um while, tente responder: “Qual condição faz o laço continuar?” e “O que fará essa condição deixar de ser verdadeira?”. Essas duas perguntas ajudam a evitar muitos erros.

10. Erros comuns no while

ERRO	O QUE ACONTECE	COMO CORRIGIR
Esquecer a atualização	Pode gerar loop infinito.	Atualize o contador/variável.
Condição nunca é verdadeira	O bloco pode nem executar.	Verifique o valor inicial.
Condição nunca fica falsa	O while não termina.	Veja o que muda dentro do laço.
Indentação errada	O código executa fora do bloco.	Mantenha os comandos indentados.
Atualizar a variável errada	A condição não muda.	Confira qual variável controla o while.

11. Como pensar antes de escrever

Use este roteiro:

1	O que precisa ser repetido?
2	Qual condição deve manter a repetição?
3	Qual variável controla essa condição?
4	O que muda a cada repetição?
5	Em qual situação o laço deve parar?

12. Exercícios de fixação

Exercício 1 — Contagem

Faça um programa que mostre os números de 1 até 10 usando while.

Exercício 2 — Números pares

Faça um programa que mostre os números pares de 2 até 20 usando while.

Exercício 3 — Cadastro

Pergunte quantos alunos serão cadastrados. Para cada aluno, peça o nome e mostre “Aluno cadastrado: nome”. Use while para controlar a quantidade.

13. Exercícios de aplicação

Exercício 4 — Menu simples

Crie um menu com as opções: 1 — Cadastrar livro; 2 — Listar livros; 3 — Sair. O menu deve continuar aparecendo até o usuário escolher 3.

Exercício 5 — Validação

Peça uma idade. Enquanto a idade for menor que 0 ou maior que 120, informe que o valor é inválido e peça novamente.

Exercício 6 — Senha

Crie um programa que peça uma senha até que o usuário digite a senha correta. Ao acertar, mostre “Acesso permitido!”.

14. Desafio — Cadastro de livros

Crie um programa que pergunte quantos livros o usuário deseja cadastrar. Para cada livro, peça:

- Título
- Autor
- Ano de publicação

Depois de cada cadastro, mostre os dados informados. Ao final, mostre “Cadastro finalizado!”. Use `while` para controlar a quantidade de cadastros.

15. Resumo para lembrar

while	repete enquanto uma condição for verdadeira
condição	determina se o laço continua
contador	pode controlar a quantidade de repetições
atualização	faz a condição mudar ao longo do programa
while + if	permite repetir e tomar decisões
while + input	permite repetir entradas e validações

Para a próxima aula: o `while` será combinado com listas. Assim, além de repetir o cadastro, poderemos guardar os dados para utilizá-los depois.

Prof. Esp Kelrison Coêlho